

Cocrates: AI를 사용하는 올바른 방법

Ep. 13 / 14

올바른 Cocrates Harness 활용

사용자와 에이전트의 공진화



AI-native Engineer를 위한 실무 가이드



 오늘 알게 될 3+1가지

 **사용자가 진화해야 한다** — Cocrates의 질문이 내 사고방식이 되는 순간

 **Cocrates Harness도 진화해야 한다** — 스킬 확장, 프로세스 커스터마이징

 **함께 자라는 피드백 루프** — 사용 경험 → 개선 → 더 나은 경험 → 성장

 **Cocrates를 사용하는 것으로 충분하지 않다**
— AI-native Engineer로서 우리 팀의 도구를 지속적으로 진화시켜야 한다

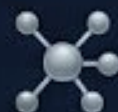


 **Cocrates** — Speaking ...

그냥 답만 주면 안 되나?



성장의 첫 신호

이 결정의 **트레이드오프**는 무엇인가?내가 놓치고 있는 **제약 조건**은 없는가?이 구조를 **타인에게 설명**할 수 있는가?

질문은 주도권을 사용자에게 돌려주기 위한 파트너의 손짓

**개인 트레이너 비유** — 트레이너가 없어도 올바른 자세로 운동할 수 있게 되는 것**Cocrates의 궁극적 목표 = 자신의 질문을 내면화하여 스스로 성장하는 사용자**



PR vs AI 검토 비교

코드 PR

한 줄 한 줄 꼼꼼히 검토



AI 결과물

"그냥 써볼까?" 하고 복붙



이중 잣대를 깨라

🎯 더 중요한 것: 구조(Architecture)의 검토

- 구조는 최종 산출물의 품질을 결정한다
- 구조 변경의 영향이 크다

? 이 구조가
요구사항을 가장
잘 충족하는가?

? 다른 구조적
대안은 없었는가?
그 대안보다 나은 이유는?

? 이 구조를
변경해야 한다면,
어디까지인가?



🏗️ 건축 설계도(구조) 검토 vs 완공된 건물(최종 산출물) 검토. 설계도가 잘못되면 건물이 흔들린다.

구조 검토 (더 중요) → 세부 설계 검토 → 최종 산출물 검토

검토는 불신이 아니라 책임감이다. 그리고 그 책임감은 구조에서부터 시작된다.

 스킬 확장

- 기본은 시작일 뿐
- 주간 보고서 → `weekly-report-authoring`
- 제안서 → `proposal-authoring`
- 팀 온보딩, 배포 검증 → **맞춤형 스킬**



"~를 생성하는 스킬을 만들어줘"
→ Ep12



프로세스 커스텀

```
~/.config/opencode/skills/  
├── ..  
├── opencode  
├── skills  
└── SKILL.md
```

- "검토 단계 너무 촘촘해" → 단계 축소
→ 단계 축소
- "내 스타일과 안 맞아"
→ 워크플로우 재구성
- 더 깊게: `cocrates.md` 프롬프트
자체 개선



팀을 위한 진화



AI-native
Engineer (팀장)



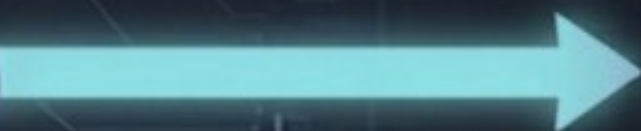
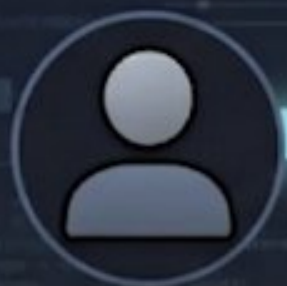
Harness



AI
팀원들

- 도구를 우리 팀 패턴에 맞게 **지속진화**
- "이 도구가 우리 팀을 더 효율적으로
만들려면 어떻게 개선할까?"

AI-native Engineer의 역할은 도구를 소비하는 데서 끝나지 않는다. 우리 팀이 일을 제대로 하는 도구로 Cocrates Harness를 지속적으로 진화시키는 것 — 이것이 Harnessing Ignorance의 실천이다.



에이전트/스킬 개선

사용 경험



진정한 AI-native Engineer



사용자 성장



더 나은 경험





오픈소스 철학



Open Source

GitHub

- 모든 것 공개: 프롬프트, 스킴, 아키텍처 문서 → GitHub
- “비결이 뭐예요?”
→ “여기 있습니다.”

[github.com/
cocrates/
cocrates.ai](https://github.com/cocrates/cocrates.ai)

플랫폼 독립성



- Claude Code, Cursor, Codex, Antigravity — 어디든 이식 가능
- 제공되는 도구를 넘어, 그 정신을 흡수하여 자신만의 하네스 구축

레고 비유



- 설명서대로 조립할 수도 있지만,
내 창의력대로 새롭게 조립할 수도 있음

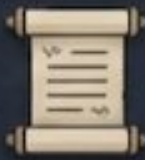


Me



Cocrates

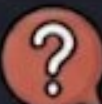
가장 바람직한 진화 방향
= 나만의 Cocrates Harness를 구축하는 것

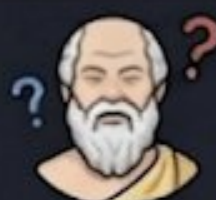
 “The unexamined code is not worth generating.” 
검토되지 않은 산출물은 생성할 가치가 없다.

Cocrates Harness = 우리 팀의 도구, 우리의 지속적 진화

 검토하라



 질문하라



 개선하라



팀장의 책임 = 무지를 활용한 지속적 진화

 3줄 요약 — 그리고 스스로에게 던지는 질문

-  **질문을 내면화하라** Cocrates의 소크라테스식 질문이 언젠가는 당신의 사고방식이 된다
-  **도구를 진화시켜라** Cocrates Harness는 AI-native Engineer와 AI 팀원이 함께 일하는 도구. 사용하는 것으로 충분하지 않다 — 우리 팀의 도구로 계속 발전시켜야 한다
-  **순환을 믿어라** 사용 경험 → 도구 개선 → 더 나은 경험 → 사용자 성장 → 다시 도구 개선으로 이어지는 선순환

— “정답이 아닌, 사고를 위한 질문” —

 나의 질문의 질문 중, 내가 이미 스스로에게 던지고 있는 것은 무엇인가?

 지금 사용하는 Cocrates Harness에서 가장 먼저 개선하고 싶은 부분은 무엇인가?

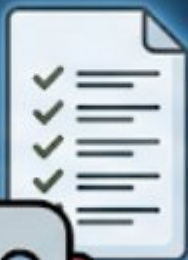
 나의 역할 Engineer로서, 나는 우리 팀의 도구를 진화시키고 있는가, 아니면 주어진 대로 소비하고 있는가?

 ‘The unexamined code’ 원칙을 Cocrates Harness에게 적용한다면, 지금 검토하지 않고 넘어간것은 무엇인가?

팀장의 책임 = 지속적 진화

핵심 메시지 요약

“Cocrates는 AI-native Engineer인 당신과 AI 팀원이 함께 일하는 도구입니다. 하지만 사용하는 것으로 충분하지 않습니다. 우리 팀이 일을 제대로 하는 도구로 Cocrates Harness를 지속적으로 진화시켜야 합니다.”



Ep14: Cocrates Harness 사용자 선언문

“더 이상 검토하지 않는 산출물은 생성하지 않겠다”

– 시리즈 전체를 관통하는 7계명으로 마무리

진정한 AI-native Engineer의 길